

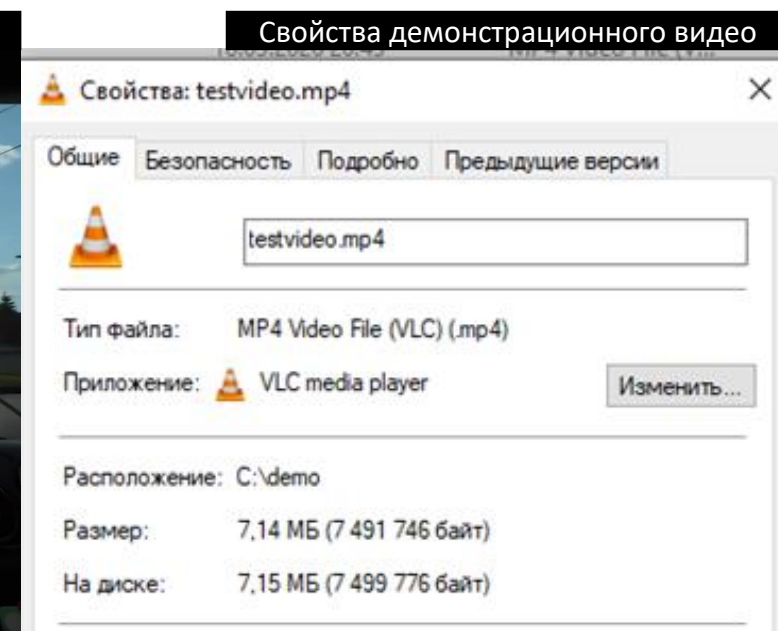
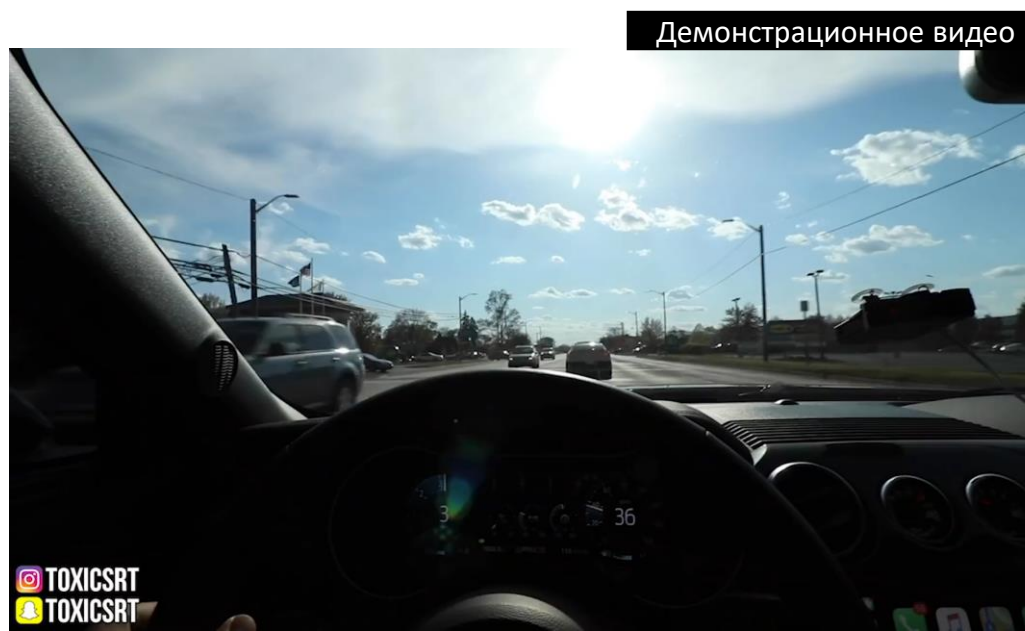
Разработка системы для быстрого декодирования видео в формате AV1

Выполнил студент группы РИС-22-2 Берсенёв Илья

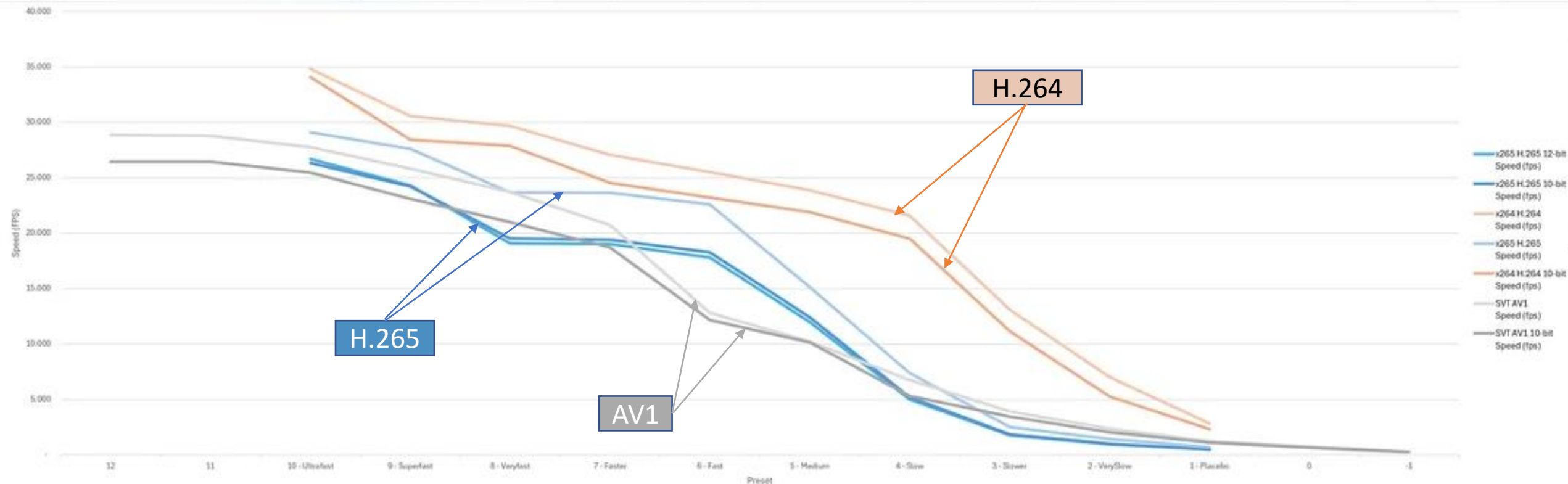
Научный руководитель В. В. Ланин

Кодирование сжимает видео в 300 раз

- $1920\text{px} * 1080\text{px} * 30\text{fps} * 3\text{channels} * 1\text{byte} * 14\text{s} = \sim 2491\text{Mb} = \sim \mathbf{2.5Gb}$
- Размер после кодирования с помощью H.264: $\sim \mathbf{7.14Mb}$
- Разница в размерах – более чем в 300 раз
- Высокая эффективность сжатия = высокая нагрузка на CPU



- H.264 (2003) -> H.265 (2012) -> AV1 (2018)
- Уменьшение скорости кодирования означает уменьшение скорости декодирования

[illegible]

Бизнес-задача Comexr

- TARe индексатор функционально эквивалентен однонаправленному видеоэнкодеру
- **Проблема:** индексация TARe в среднем в 24 раза быстрее чем видеodeкодирование
- **Следствие:** узкое место в пайплайне системы
- **Цель:** ускорить декодирование видео чтобы устранить узкое место



Гипотеза, объект и субъект исследования

Объект исследования

DAV1D – AV1 видеodeкодер
(spec v1.0.0 errata 1)

Субъект исследования

Производительность видеodeкодирования в выходном разрешении 64 пикселя, в однопоточном режиме, без использования NVIDIA CUDA и SIMD

Цель исследования: разработать и применить оптимизации видеodeкодера AV1 для выходного разрешения 64 пикселя, спроектировать архитектуру системы декодирования с учетом максимальной производительности

Гипотеза исследования: Алгоритмические оптимизации видеodeкодера и архитектурные оптимизации системы могут повысить совокупную производительность на более чем 30%

Ограничения исследования

- Выходное разрешение видео: 64 пикселя
- Однопоточный режим выполнения
- Отсутствие оптимизаций, специфичных для архитектуры
 - NVIDIA CUDA
 - SIMD (intel AVX, SSSE, SSE, etc.)
- Сохранение качества видео выше 15db по метрике PSNR

Анализ аналогов

- Не было выявлено решений данной или аналогичной задачи, так как потеря качества в современных кодеках недопустима
 - Исключение – кодеки для embedded, но в данных исследованиях снижается CPU нагрузка, а не ускоряется кодек
- Было выявлено 3 видеodeкодера: DAV1D, RAV1D, libaom
 - Однопоточный режим, без SIMD, CPU: intel i5-12400f

Сравнение видеodeкодеров AV1

<u>Видеodeкодер</u>	<u>Скорость декодирования получасового видео, в секундах</u>	<u>Язык реализации</u>
libaom	679,788	C
DAV1D	374,877	C
RAV1D	555,763	Rust, C

Анализ предметной области

Нефункциональные требования

- Нагрузка при обработке нескольких запросов должна линейно масштабироваться при превышении загруженности процессорных мощностей чтобы снизить задержки.
- Декодер AV1 должен быть собран под архитектуру generic и не использовать Nvidia CUDA.
- Оптимизированная система должна проходить путь загрузка-декодирование быстрее наивной реализации (см. главу 2) не менее чем на **30%**.
- Оптимизированный видеodeкодер должен декодировать видео быстрее чем его неоптимизированная реализация не менее чем на **20%**.

Функциональные требования

- Система должна декодировать видео в формате AV1
- Система должна иметь возможность загрузки видео пользователем
- Система должна хранить метаданные о загруженных видеофайлах и статусах задач в реляционной базе данных.
- Система должна уведомлять пользователя о завершении задачи декодирования.

Анализ стандарта AV1

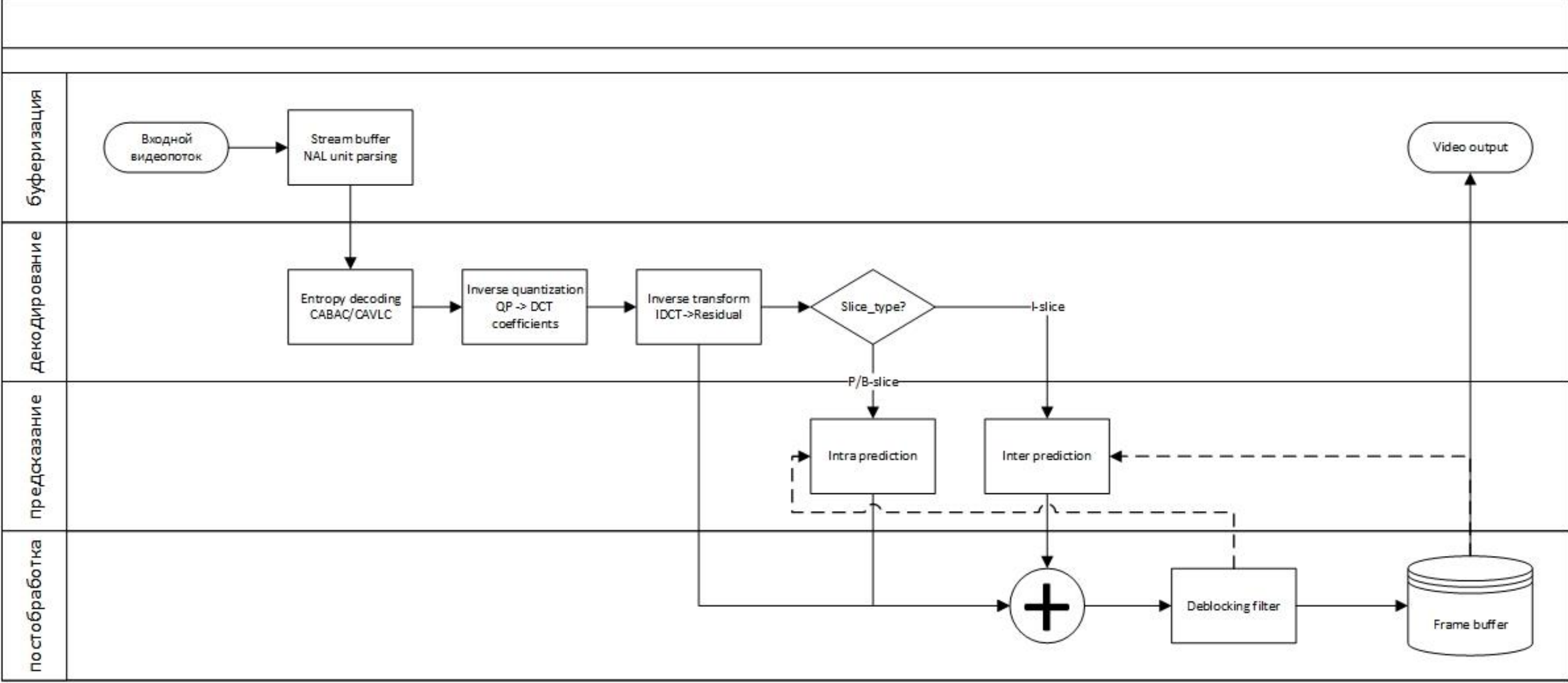
Документы:

- AV1 Bitstream & Decoding Specification v1.0.0-errata
- ITU-T Recommendation H.264



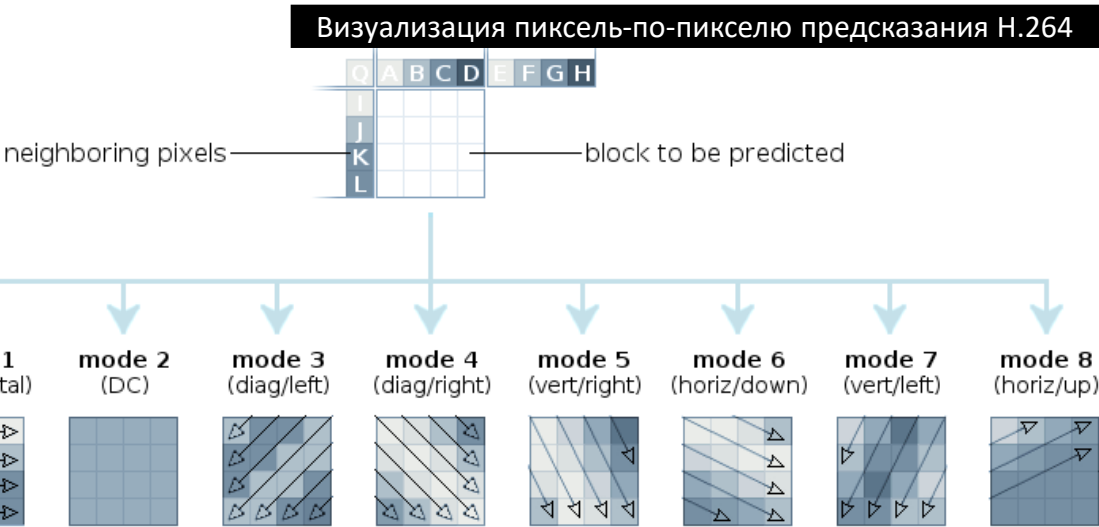
Упрощённый процесс видеodeкодирования на примере H.264

Упрощённый процесс декодирования

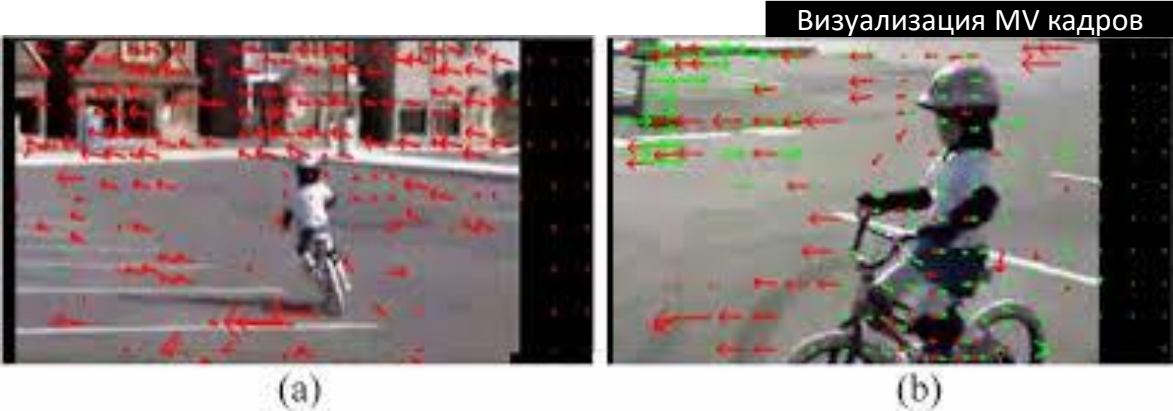


Способы предсказания

- Intra – межкадровое
- Inter – внутрикадровое

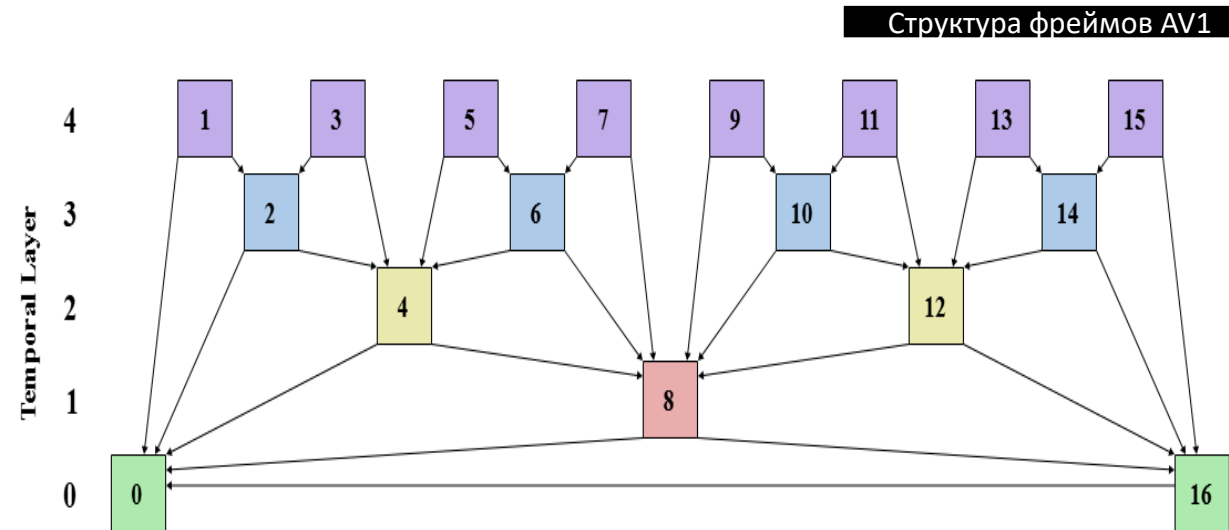
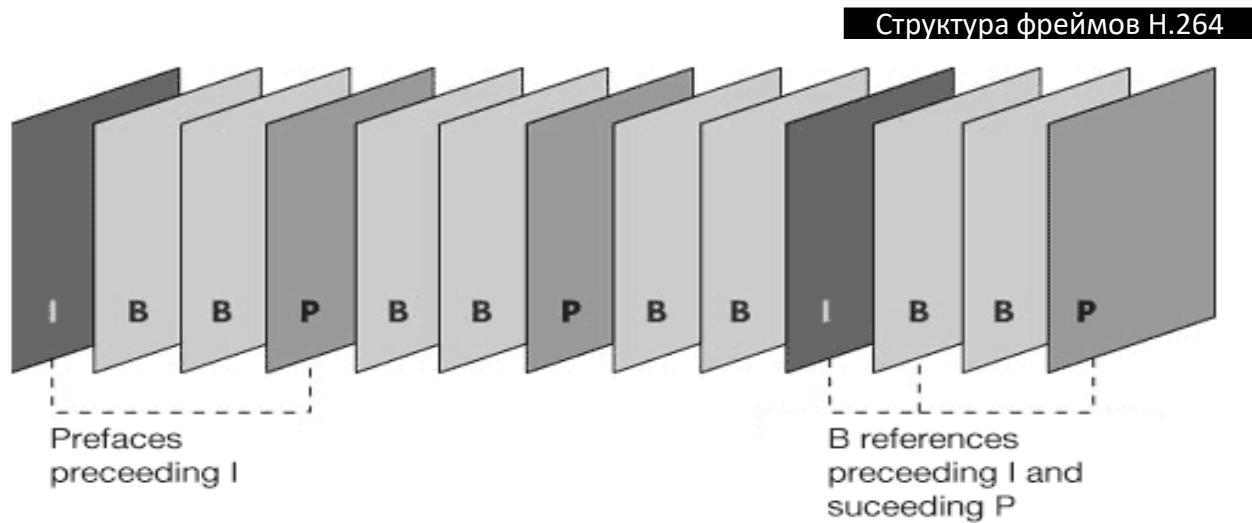


AVC/H.264 intra prediction modes



Inter предсказание

- Фрейм = кадр
- Фрейм делится на I, P, B фреймы (H.264)
- Фрейм делится на Golden фреймы (I), B фреймы (AV1)



Inter предсказание: Motion Vectors

- Motion Vectors (MV) – метод кодирования движущихся частей видео с помощью дельты x и дельты y кадра по сравнению с опорным



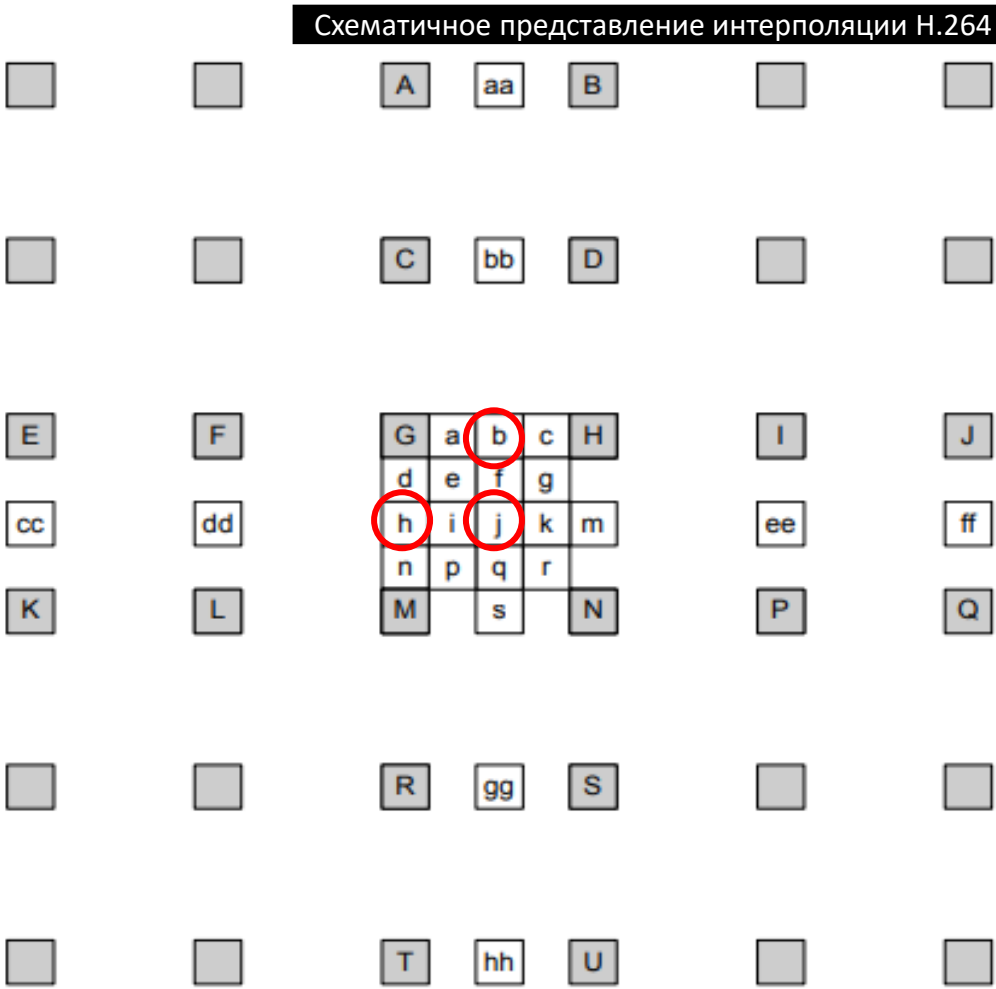
Визуализация MV стокового видео



Inter предсказание: MV: Интерполяция H.264

• Пример интерполяции для H.264:

- $b = (E - 5 * F + 20 * G + 20 * H - 5 * I + J) / 32 \leftarrow \text{71 такт}$
- $h = (A - 5 * C + 20 * G + 20 * M - 5 * R + T) / 32 \leftarrow \text{71 такт}$
- $j = ($
 $(cc - 5 * dd + 20 * h +$
 $20 * m - 5 * ee + ff) +$
 $(aa - 5 * bb + 20 * b +$
 $20 * s - 5 * gg + hh)$
 $) / (32 * 2) \leftarrow \text{очень много тактов}$



Inter предсказание: MV: Интерполяция AV1

- 8-tap (AV1) вместо 6-tap (H.264)
- 96 наборов динамических коэффициентов вместо одного статического набора в H.264

Наборы коэффициентов интерполяции AV1

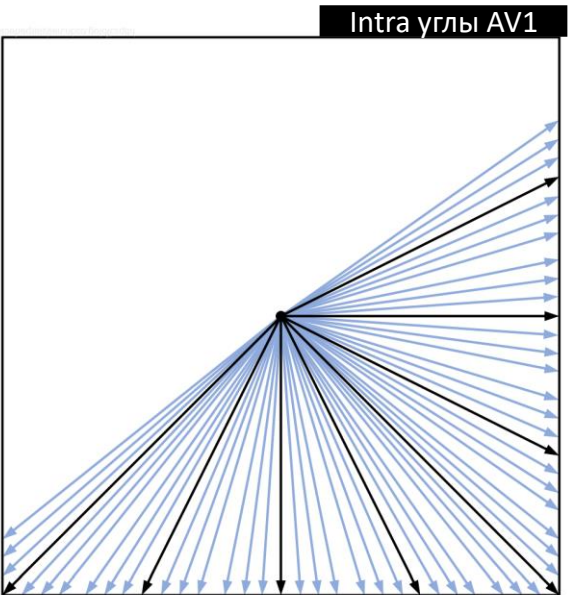
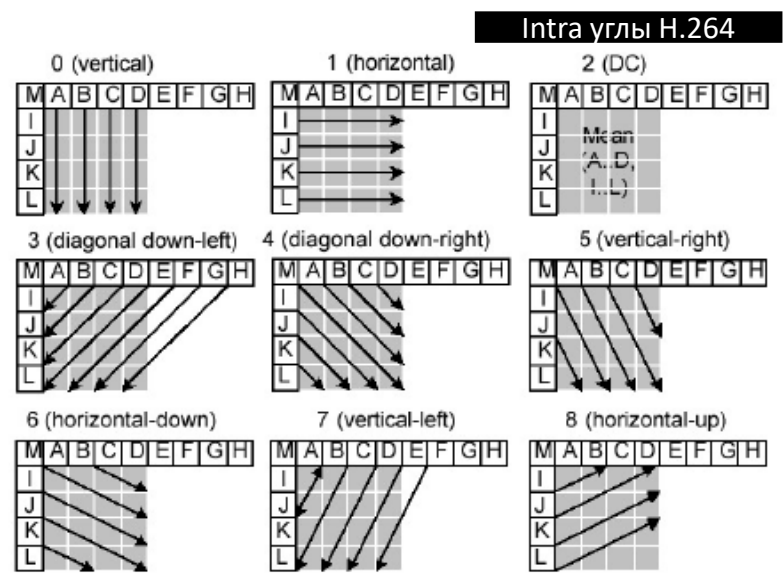
Position	Regular	Smooth	Sharp	Bilinear	4-Tap Regular	4-Tap Smooth
0/16	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}
1/16	{0,2,-6,126,8,-2,0,0}	{0,2,28,62,34,2,0,0}	{-2,2,-6,126,8,-2,2,0}	{0,0,0,120,8,0,0,0}	{0,0,-4,126,8,-2,0,0}	{0,0,30,62,34,2,0,0}
2/16	{0,2,-10,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}	{-2,6,-12,124,16,-6,4,-2}	{0,0,0,112,16,0,0,0}	{0,0,-8,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}
3/16	{0,2,-12,116,28,-8,2,0}	{0,0,22,62,40,4,0,0}	{-2,8,-18,120,26,-10,6,-2}	{0,0,0,104,24,0,0,0}	{0,0,-10,116,28,-6,0,0}	{0,0,22,62,40,4,0,0}
4/16	{0,2,-14,110,38,-10,2,0}	{0,0,20,60,42,6,0,0}	{-4,10,-22,116,38,-14,6,-2}	{0,0,0,96,32,0,0,0}	{0,0,-10,110,38,-8,0,0}	{0,0,20,60,42,6,0,0}
5/16	{0,2,-14,102,48,-12,2,0}	{0,0,18,58,44,8,0,0}	{-4,10,-22,108,48,-18,8,-2}	{0,0,0,88,40,0,0,0}	{0,0,-12,102,48,-10,0,0}	{0,0,18,58,44,8,0,0}
6/16	{0,2,-16,94,58,-12,2,0}	{0,0,16,56,46,10,0,0}	{-4,10,-24,100,60,-20,8,-2}	{0,0,0,80,48,0,0,0}	{0,0,-14,94,58,-10,0,0}	{0,0,16,56,46,10,0,0}
7/16	{0,2,-14,84,66,-12,2,0}	{0,-2,16,54,48,12,0,0}	{-4,10,-24,90,70,-22,10,-2}	{0,0,0,72,56,0,0,0}	{0,0,-12,84,66,-10,0,0}	{0,0,14,54,48,12,0,0}
8/16	{0,2,-14,76,76,-14,2,0}	{0,-2,14,52,52,14,-2,0}	{-4,12,-24,80,80,-24,12,-4}	{0,0,0,64,64,0,0,0}	{0,0,-12,76,76,-12,0,0}	{0,0,12,52,52,12,0,0}
9/16	{0,2,-12,66,84,-14,2,0}	{0,0,12,48,54,16,-2,0}	{-2,10,-22,70,90,-24,10,-4}	{0,0,0,56,72,0,0,0}	{0,0,-10,66,84,-12,0,0}	{0,0,12,48,54,14,0,0}
10/16	{0,2,-12,58,94,-16,2,0}	{0,0,10,46,56,16,0,0}	{-2,8,-20,60,100,-24,10,-4}	{0,0,0,48,80,0,0,0}	{0,0,-10,58,94,-14,0,0}	{0,0,10,46,56,16,0,0}
11/16	{0,2,-12,48,102,-14,2,0}	{0,0,8,44,58,18,0,0}	{-2,8,-18,48,108,-22,10,-4}	{0,0,0,40,88,0,0,0}	{0,0,-10,48,102,-12,0,0}	{0,0,8,44,58,18,0,0}
12/16	{0,2,-10,38,110,-14,2,0}	{0,0,6,42,60,20,0,0}	{-2,6,-14,38,116,-22,10,-4}	{0,0,0,32,96,0,0,0}	{0,0,-8,38,110,-12,0,0}	{0,0,6,42,60,20,0,0}
13/16	{0,2,-8,28,116,-12,2,0}	{0,0,4,40,62,22,0,0}	{-2,6,-10,26,120,-18,8,-2}	{0,0,0,24,104,0,0,0}	{0,0,-6,28,116,-10,0,0}	{0,0,4,40,62,22,0,0}
14/16	{0,0,-4,18,122,-10,2,0}	{0,0,4,36,62,26,0,0}	{-2,4,-6,16,124,-12,6,-2}	{0,0,0,16,112,0,0,0}	{0,0,-4,18,122,-8,0,0}	{0,0,4,36,62,26,0,0}
15/16	{0,0,-2,8,126,-6,2,0}	{0,0,2,34,62,28,2,0}	{0,2,-2,8,126,-6,2,-2}	{0,0,0,8,120,0,0,0}	{0,0,-2,8,126,-4,0,0}	{0,0,2,34,62,30,0,0}

Intra предсказание

Intra предсказание представляет из себя вычисление пикселя по пикселю.

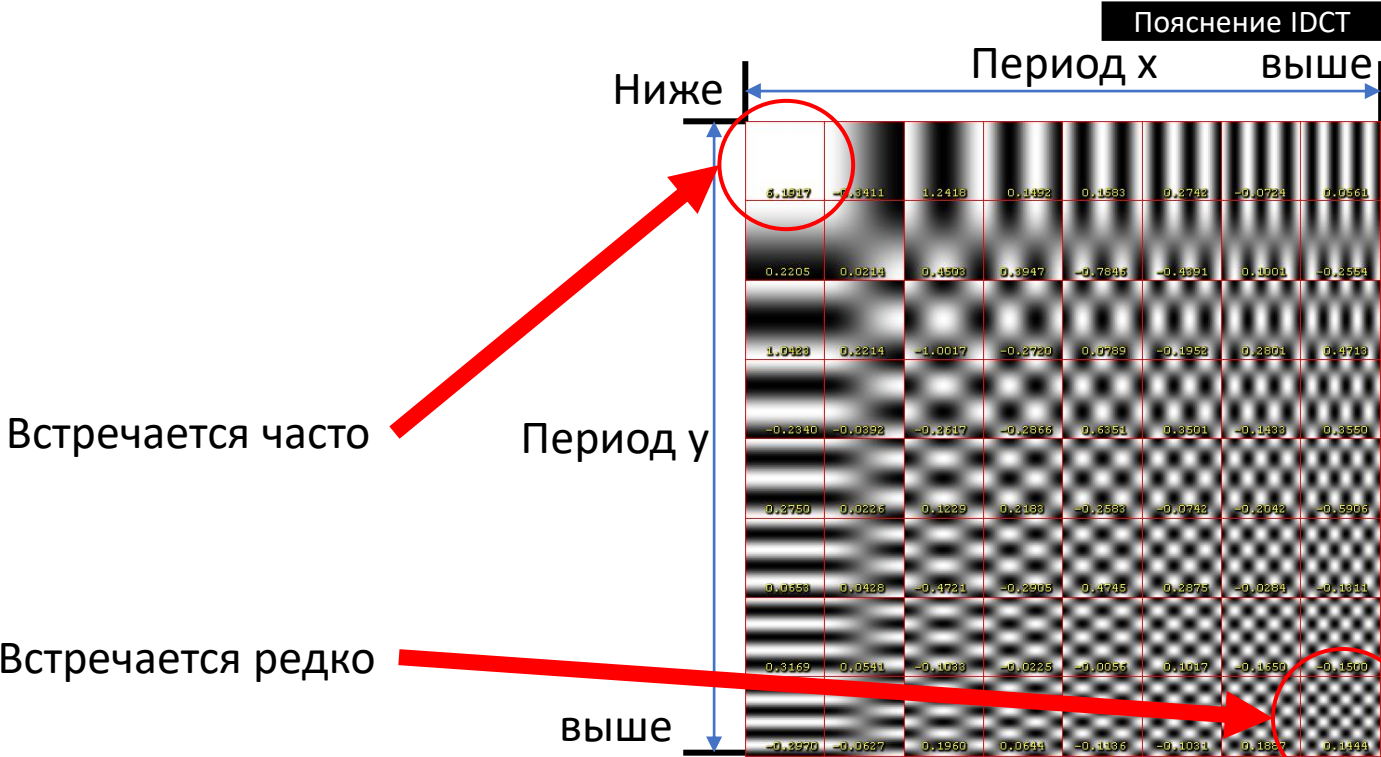
H.264: 8 направлений

AV1: 56 направлений, рекурсивное предсказание



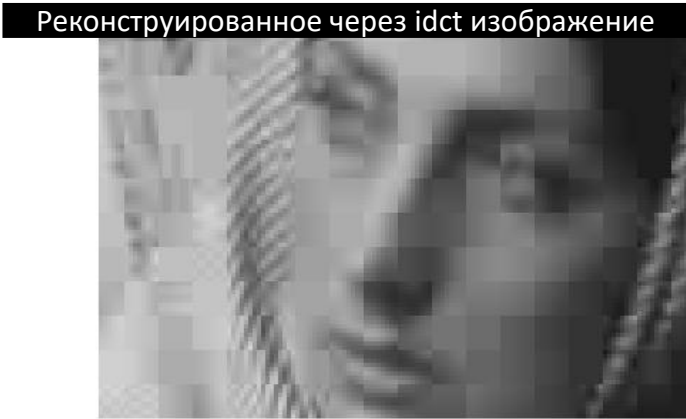
Intra предсказание: обратная дискретная косинусная трансформация

Обратная дискретная косинусная трансформация (IDCT) представляет из себя реконструкцию текстуры 16*16 с помощью периодов двух синусоид. Чем больше период, тем реже текстура встречается в видео

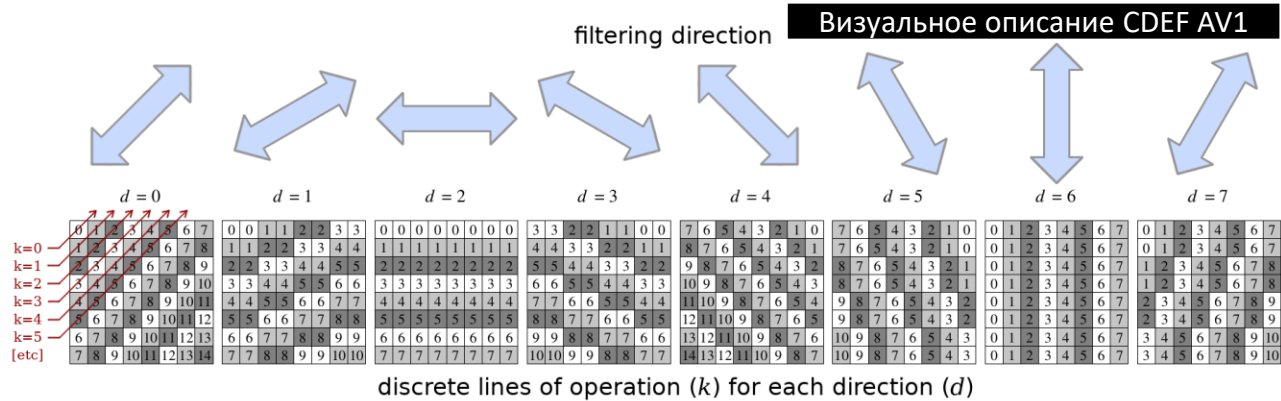


Intra предсказание: CDEF, DF

Deblocking filter: фильтр постобработки, устраняющий артефакты блочности после IDCT



Constrained directional enhancement filter: фильтр постобработки, применяющий 8 блочных фильтров направлений к каждому блоку фрейма



Выводы по анализу

Узкие места:

- IDCT
- DF
- CDEF
- Интерполяция

Проектирование

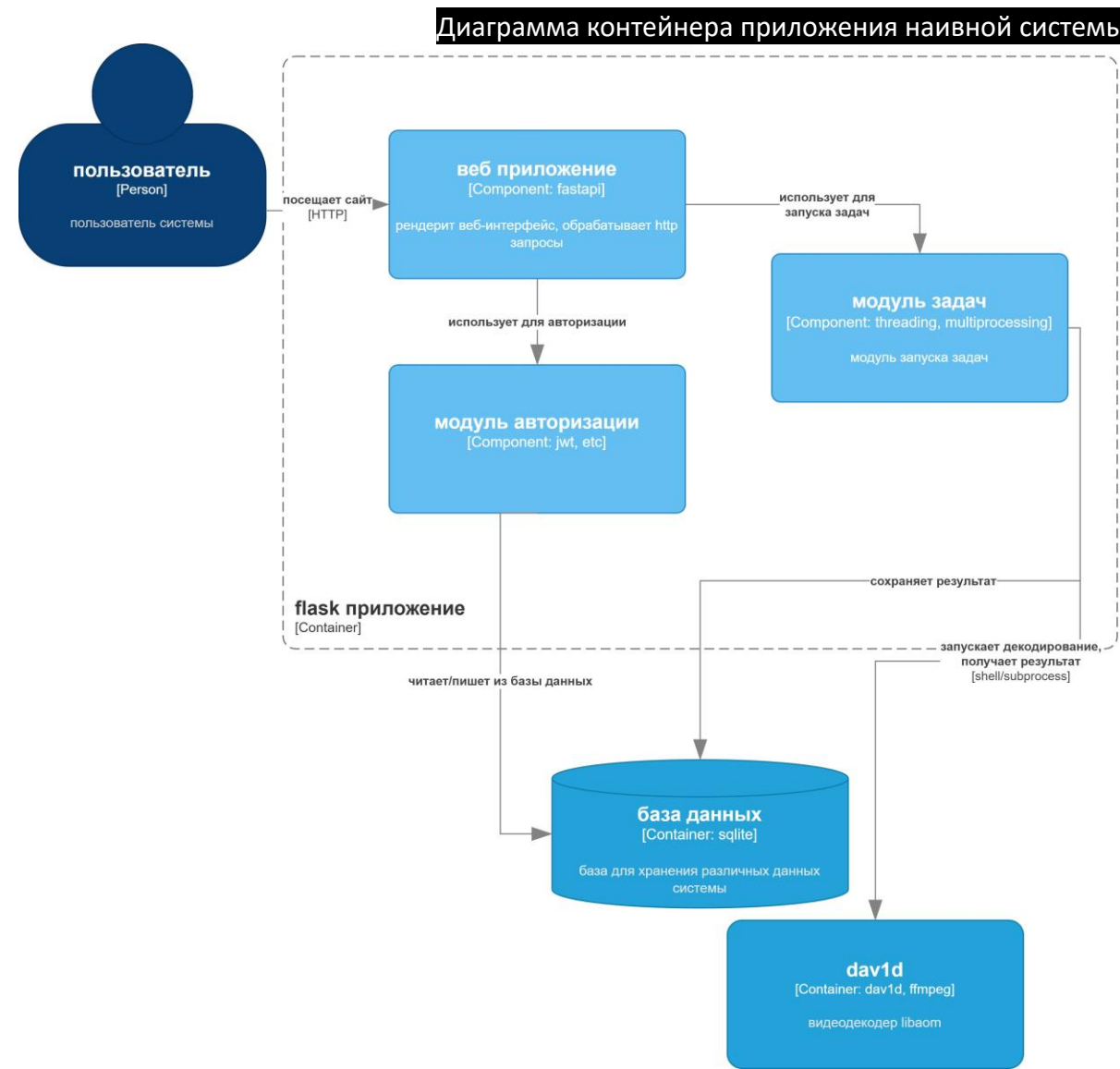
Наивная система

× Не имеет потенциала к масштабированию и оптимизации



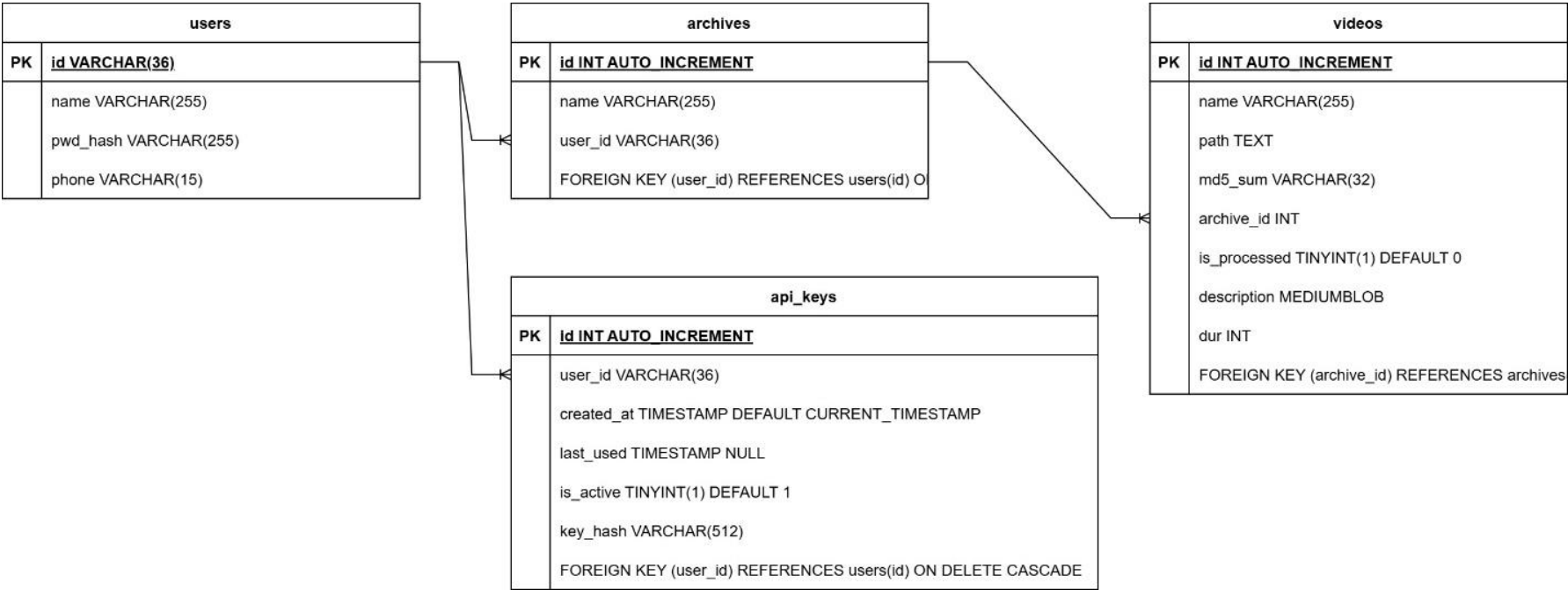
Наивная система: приложение

- Модуль авторизации
 - jwt
- Модуль задач
 - Threading, multiprocessing
- Веб-приложение
 - Fastapi



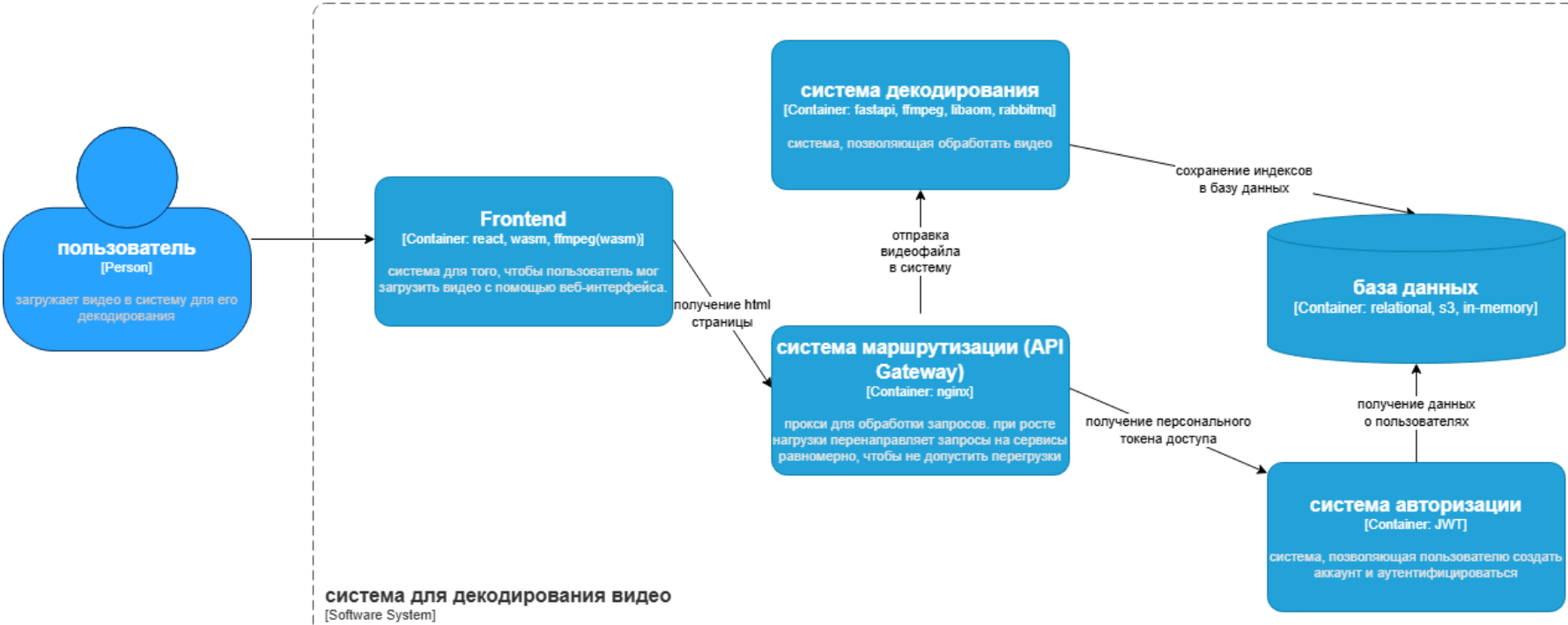
Наивная система: база данных

Структура базы одина для наивной и оптимизированной систем



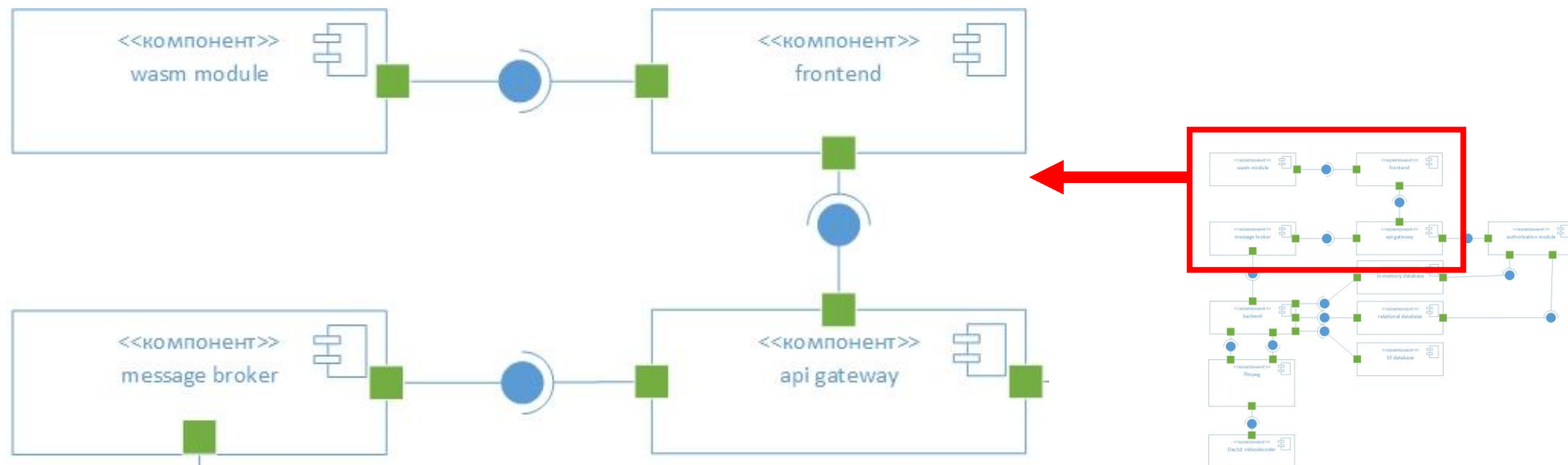
Оптимизированная система

- Frontend
 - Wasm, ffmpeg
- Система декодирования
 - Fastapi, ffmpeg, dav1d
- База данных
 - S3(Ceph), postgresql, Redis
- Система маршрутизации
 - Fastapi, websockets
- Система авторизации
 - JWT



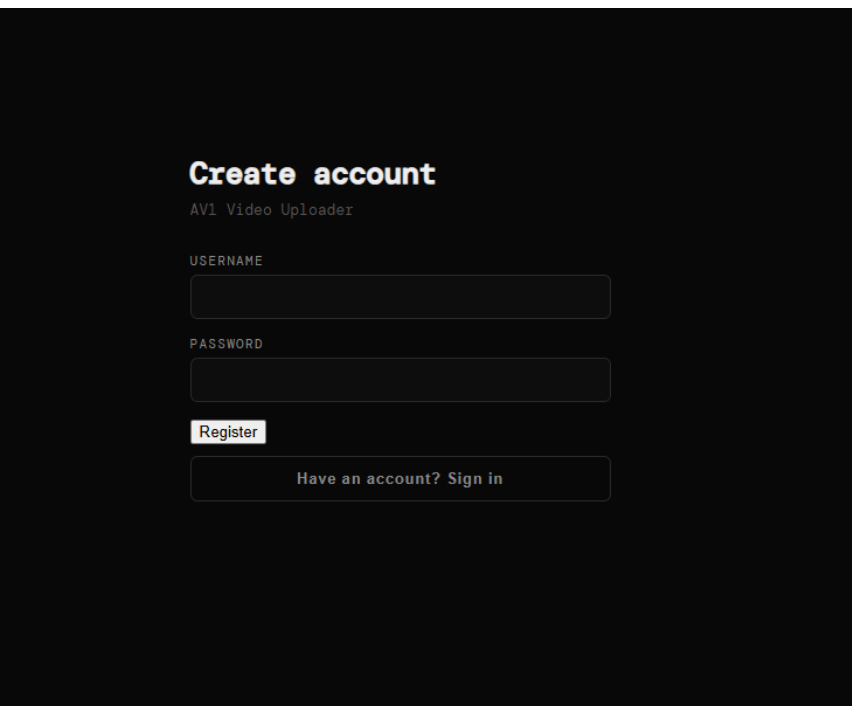
Оптимизированная система: фронтенд

- React
- FFmpeg(WASM)
 - Необходимо для отделения аудио от видео на стороне клиента



Оптимизированная система: фронтенд: дизайн

Дизайн фронтенда состоит из 2 панелей: авторизация и загрузка



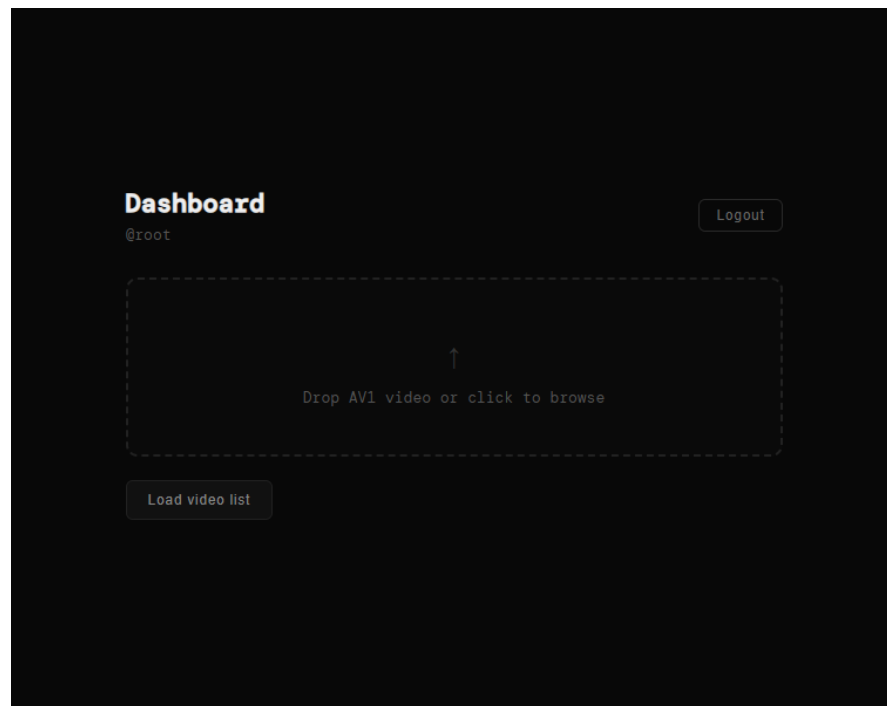
Create account
AV1 Video Uploader

USERNAME

PASSWORD

Register

Have an account? Sign in



Dashboard
@root Logout

↑

Drop AV1 video or click to browse

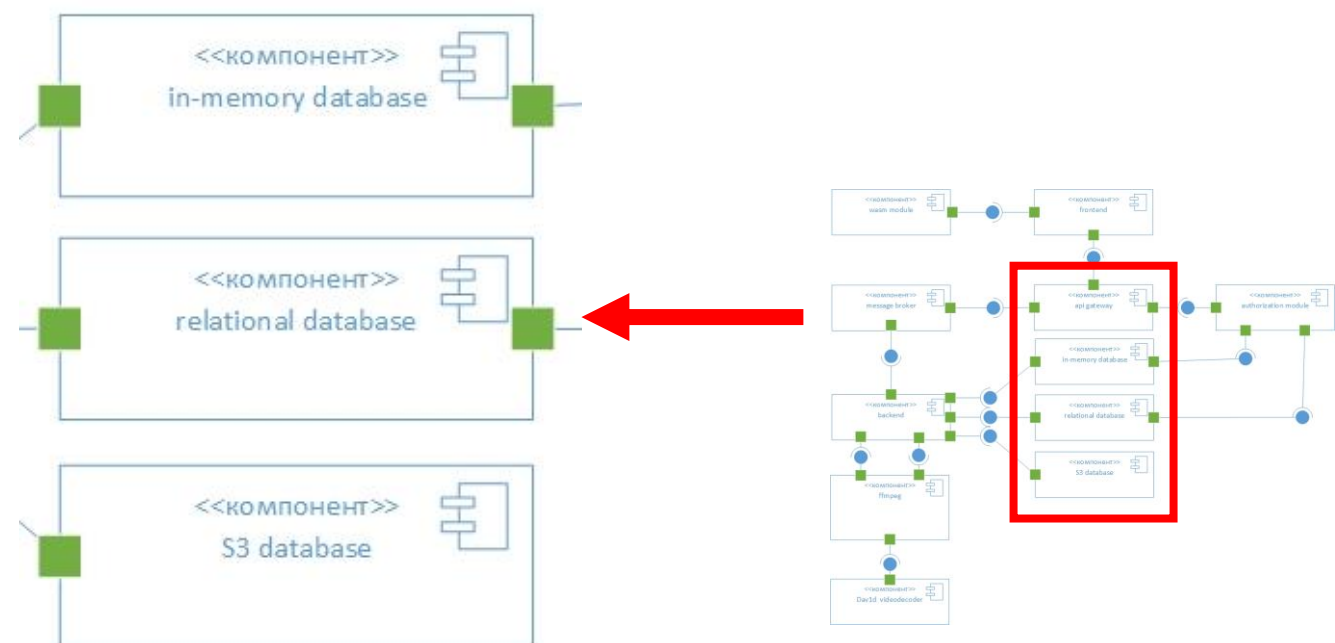
Load video list

Оптимизированная система: базы данных

Базы данных:

- High latency реляционная база данных – Postgresql
- Low latency нереляционная база данных – Redis
- High latency нереляционная

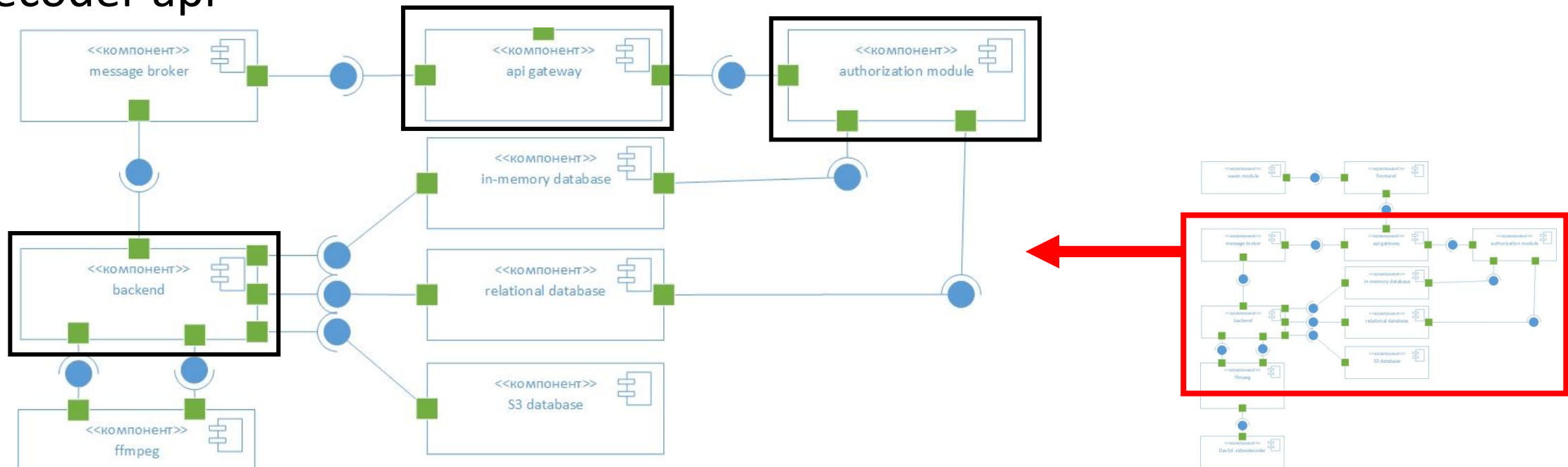
база данных: S3(Ceph)



Оптимизированная система: бэкенд

Компоненты:

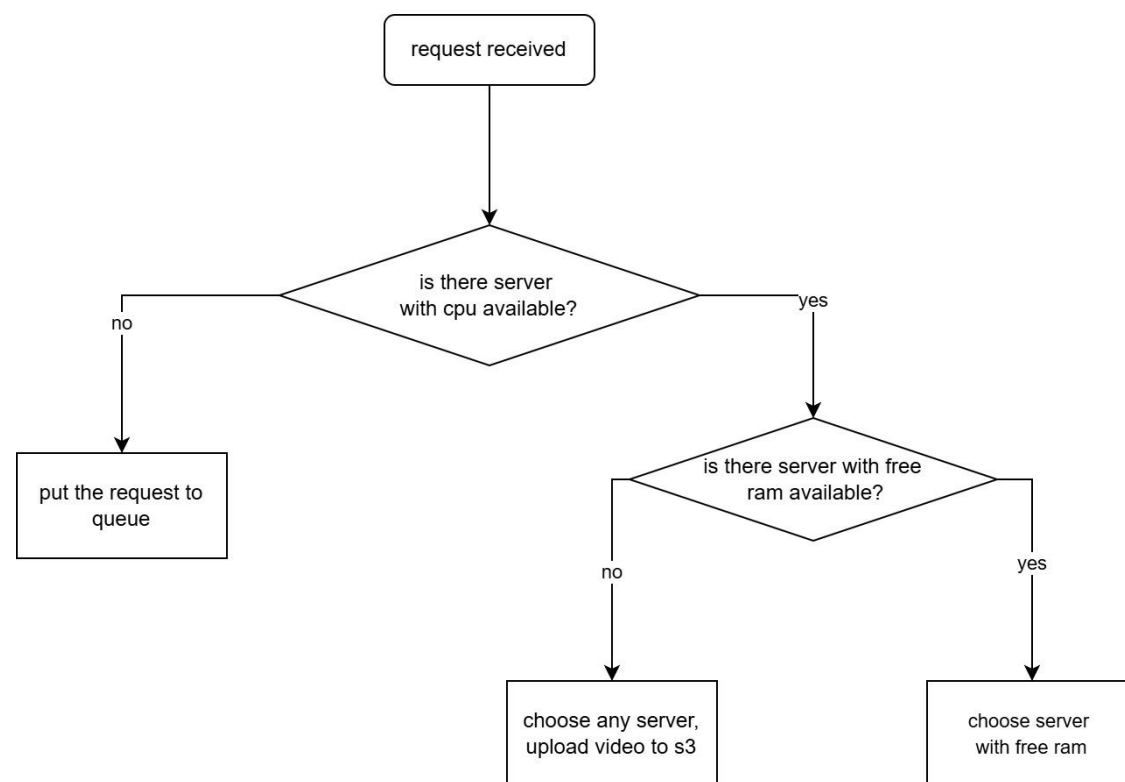
- Gateway
- Common api
- Decoder api



Оптимизированная система: Gateway

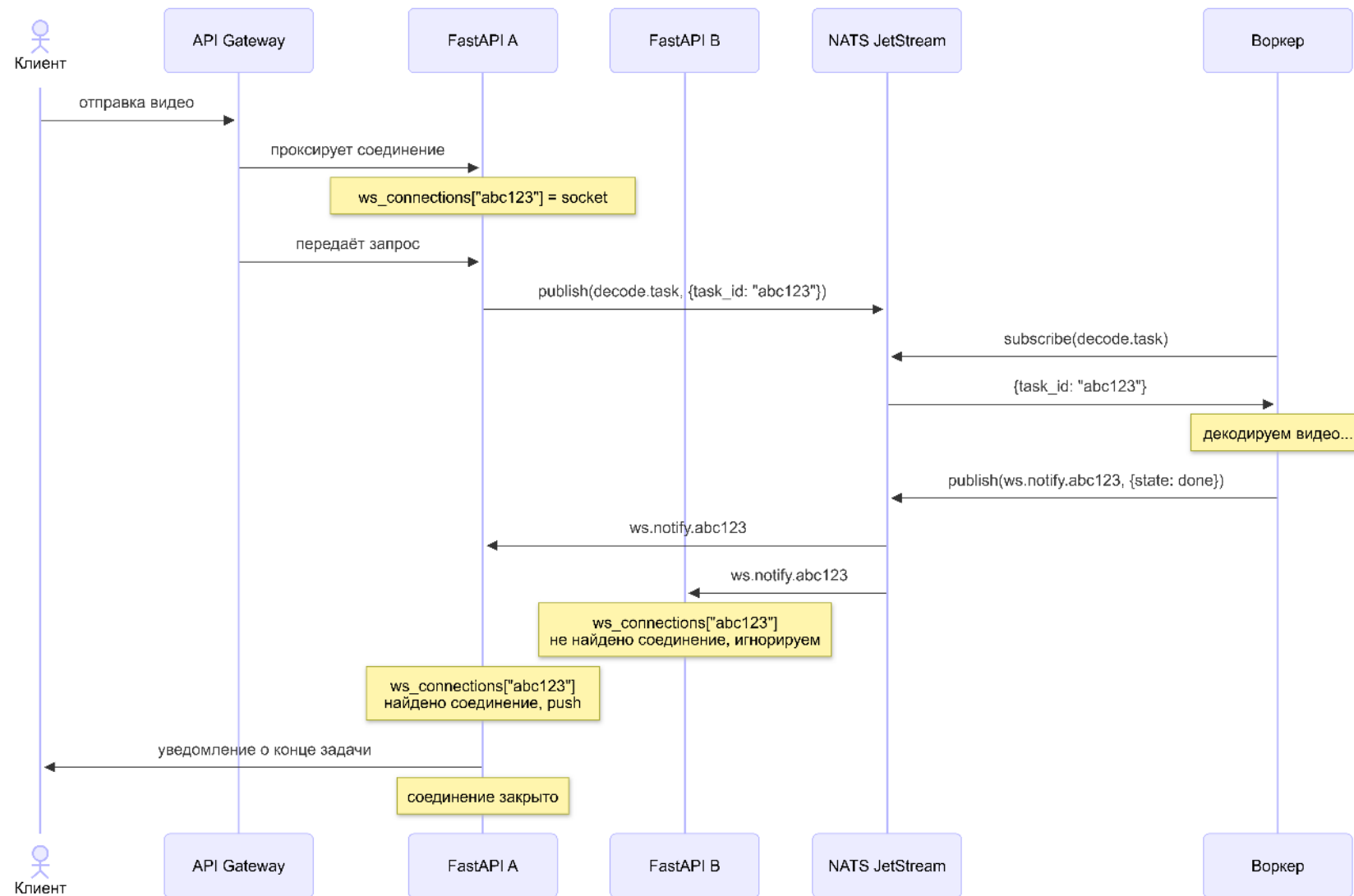
Система маршрутизации (Gateway) – прокси и балансировщик.

- Собирает информацию от реплик decoder-арі через вебсокет
- Проксирует запросы авторизации на common-арі
- В зависимости от нагрузки на репликах decoder-арі выбирает наименее загруженную реплику



Оптимизированная система: Gateway

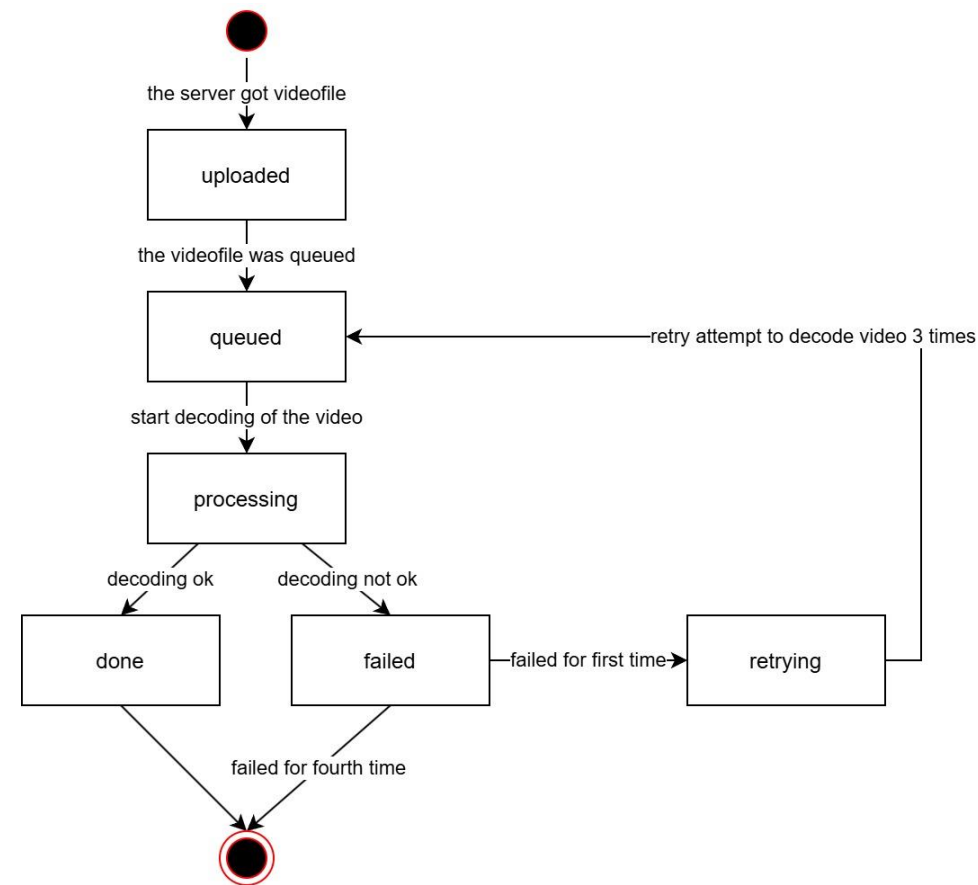
Система маршрутизации также выступает держателем всех WebSocket соединений с клиентами, уведомляя пользователя о конце операции



Оптимизированная система: Decoder api

Функции:

- Декодирование видео
 - Из оперативной памяти
 - С диска
- Уведомление пользователя о конце операции (через очередь JetStream)



Реализация: оптимизации DAV1D



DAV1D: CDEF, LF

Идея: полностью отключить CDEF, LF, так как на низком разрешении артефакты блочности будут не видны.

Результат: -26% скорости декодирования, артефакты не видны невооруженным глазом.

DAV1D: CDEF, LF



DAV1D: 6-tap интерполяция

Идея: сократить количество фильтров интерполяции с 8 до 6. Это можно сделать так как в большинстве случаев первый и восьмой фильтры равны нулю.

Результат: -30.5% времени декодирования, начали копиться небольшие артефакты

Position	Regular	Smooth	Sharp	Bilinear	4-Tap Regular	4-Tap Smooth
0/16	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}
1/16	{0,2,-6,126,8,-2,0,0}	{0,2,28,62,34,2,0,0}	{-2,2,-6,126,8,-2,2,0}	{0,0,0,120,8,0,0,0}	{0,0,-4,126,8,-2,0,0}	{0,0,30,62,34,2,0,0}
2/16	{0,2,-10,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}	{-2,6,-12,124,16,-6,4,-2}	{0,0,0,112,16,0,0,0}	{0,0,-8,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}
3/16	{0,2,-12,116,28,8,2,0}	{0,0,22,62,40,4,0,0}	{-2,8,-18,120,26,-10,6,-2}	{0,0,0,104,24,0,0,0}	{0,0,-10,116,28,8,0,0}	{0,0,22,62,40,4,0,0}
4/16	{0,2,-14,110,38,10,2,0}	{0,0,20,60,42,6,0,0}	{-4,10,-22,116,38,-14,6,-2}	{0,0,0,96,32,0,0,0}	{0,0,-10,110,38,8,0,0}	{0,0,20,60,42,6,0,0}
5/16	{0,2,-14,102,48,12,2,0}	{0,0,18,58,44,8,0,0}	{-4,10,-22,108,48,-18,8,-2}	{0,0,0,88,40,0,0,0}	{0,0,-12,102,48,10,0,0}	{0,0,18,58,44,8,0,0}
6/16	{0,2,-16,94,58,-2,2,0}	{0,0,16,56,46,10,0,0}	{-4,10,-24,100,60,-20,8,-2}	{0,0,0,80,48,0,0,0}	{0,0,-14,94,58,-2,0,0}	{0,0,16,56,46,10,0,0}
7/16	{0,2,-14,84,66,-2,2,0}	{0,0,2,16,54,48,12,0,0}	{-4,10,-24,90,70,-22,10,-2}	{0,0,0,72,56,0,0,0}	{0,0,-12,84,66,-2,0,0}	{0,0,14,54,48,12,0,0}
8/16	{0,2,-14,76,76,-4,2,0}	{0,0,2,14,52,52,14,-2,0}	{-4,12,-24,80,80,-24,12,-4}	{0,0,0,64,64,0,0,0}	{0,0,-12,76,76,-2,0,0}	{0,0,12,52,52,14,0,0}
9/16	{0,2,-12,66,84,-4,2,0}	{0,0,12,48,54,16,-2,0}	{-2,10,-22,70,90,-24,10,-4}	{0,0,0,56,72,0,0,0}	{0,0,-10,66,84,-2,0,0}	{0,0,12,48,54,16,0,0}
10/16	{0,2,-12,58,94,-6,2,0}	{0,0,10,46,56,16,0,0}	{-2,8,-20,60,100,-24,10,-4}	{0,0,0,48,80,0,0,0}	{0,0,-10,58,94,-4,0,0}	{0,0,10,46,56,16,0,0}
11/16	{0,2,-12,48,102,-14,2,0}	{0,0,8,44,58,18,0,0}	{-2,8,-18,48,108,-22,10,-4}	{0,0,0,40,88,0,0,0}	{0,0,-10,48,102,-12,0,0}	{0,0,8,44,58,18,0,0}
12/16	{0,2,-10,38,110,-14,2,0}	{0,0,6,42,60,20,0,0}	{-2,6,-14,38,116,-22,10,-4}	{0,0,0,32,96,0,0,0}	{0,0,-8,38,110,-12,0,0}	{0,0,6,42,60,20,0,0}
13/16	{0,2,-8,28,116,-12,2,0}	{0,0,4,40,62,22,0,0}	{-2,6,-10,26,120,-18,8,-2}	{0,0,0,24,104,0,0,0}	{0,0,-6,28,116,-10,0,0}	{0,0,4,40,62,22,0,0}
14/16	{0,0,-4,18,122,-10,2,0}	{0,0,4,36,62,26,0,0}	{-2,4,-6,16,124,-12,6,-2}	{0,0,0,16,112,0,0,0}	{0,0,-4,18,122,-8,0,0}	{0,0,4,36,62,26,0,0}
15/16	{0,0,-2,8,126,-6,2,0}	{0,0,2,34,62,28,0,0}	{0,0,-2,8,126,-6,2,-2}	{0,0,0,8,120,0,0,0}	{0,0,-2,8,126,-4,0,0}	{0,0,2,34,62,30,0,0}

DAV1D: 6-тар интерполяция



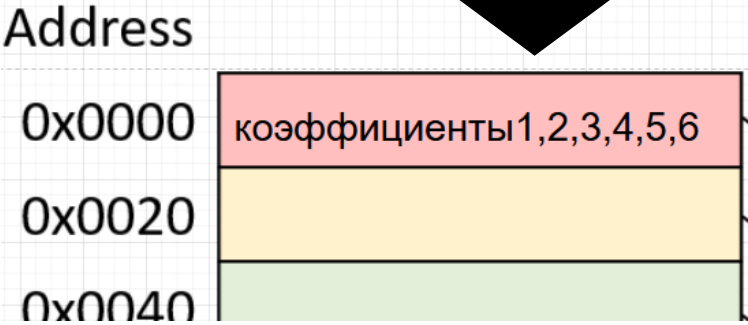
DAV1D: сокращение набора коэффициентов интерполяции

Идея: увеличить локальность данных, сократив все наборы коэффициентов до 6 и выровняв данные чтобы они гарантированно попали в одну линию процессорного кеша.

Результат: -32.5% времени декодирования, незаметные глазу артефакты



Position	Regular	Smooth	Sharp	Bilinear	4-Tap Regular	4-Tap Smooth
0/16	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}	{0,0,0,128,0,0,0,0}
1/16	{0,2,-6,126,8,-2,0,0}	{0,2,28,62,34,2,0,0}	{-2,2,-6,126,8,-2,2,0}	{0,0,0,120,8,0,0,0}	{0,0,-4,126,8,-2,0,0}	{0,0,30,62,34,2,0,0}
2/16	{0,2,-10,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}	{-2,6,-12,124,16,-6,4,-2}	{0,0,0,112,16,0,0,0}	{0,0,-8,122,18,-4,0,0}	{0,0,26,62,36,4,0,0}
3/16	{0,2,-12,116,28,-8,2,0}	{0,0,22,62,40,4,0,0}	{-2,8,-18,120,26,-10,6,-2}	{0,0,0,104,24,0,0,0}	{0,0,-10,116,28,-6,0,0}	{0,0,22,62,40,4,0,0}
4/16	{0,2,-14,110,38,-10,2,0}	{0,0,20,60,42,6,0,0}	{-4,10,-22,116,38,-14,6,-2}	{0,0,0,96,32,0,0,0}	{0,0,-10,110,38,-8,0,0}	{0,0,20,60,42,6,0,0}
5/16	{0,2,-14,102,48,-12,2,0}	{0,0,18,58,44,8,0,0}	{-4,10,-22,108,48,-18,8,-2}	{0,0,0,88,40,0,0,0}	{0,0,-12,102,48,-10,0,0}	{0,0,18,58,44,8,0,0}
6/16	{0,2,-16,94,58,-12,2,0}	{0,0,16,56,46,10,0,0}	{-4,10,-24,100,58,-20,8,-2}	{0,0,0,80,48,0,0,0}	{0,0,-14,94,58,-10,0,0}	{0,0,16,56,46,10,0,0}
7/16	{0,2,-14,84,66,-12,2,0}	{0,0,2,16,54,48,12,0,0}	{-4,10,-24,90,70,-22,10,-2}	{0,0,0,72,56,0,0,0}	{0,0,-12,84,66,-10,0,0}	{0,0,14,54,48,12,0,0}
8/16	{0,2,-14,76,76,-14,2,0}	{0,0,2,14,52,52,14,-2,0}	{-4,12,-24,80,80,-24,12,-4}	{0,0,0,64,64,0,0,0}	{0,0,-12,76,76,-12,0,0}	{0,0,12,52,52,12,0,0}
9/16	{0,2,-12,66,84,-14,2,0}	{0,0,12,48,54,16,-2,0}	{-2,10,-22,70,90,-24,10,-4}	{0,0,0,56,72,0,0,0}	{0,0,-10,66,84,-12,0,0}	{0,0,12,48,54,14,0,0}
10/16	{0,2,-12,58,94,-16,2,0}	{0,0,10,46,56,16,0,0}	{-2,8,-20,60,100,-24,10,-4}	{0,0,0,48,80,0,0,0}	{0,0,-10,58,94,-14,0,0}	{0,0,10,46,56,16,0,0}
11/16	{0,2,-12,48,102,-14,2,0}	{0,0,8,44,58,18,0,0}	{-2,8,-18,48,108,-22,10,-4}	{0,0,0,40,88,0,0,0}	{0,0,-10,48,102,-12,0,0}	{0,0,8,44,58,18,0,0}
12/16	{0,2,-10,38,110,-14,2,0}	{0,0,6,42,60,20,0,0}	{-2,6,-14,38,116,-22,10,-4}	{0,0,0,32,96,0,0,0}	{0,0,-8,38,110,-12,0,0}	{0,0,6,42,60,20,0,0}
13/16	{0,2,-8,28,116,-12,2,0}	{0,0,4,40,62,22,0,0}	{-2,6,-10,26,120,-18,8,-2}	{0,0,0,24,104,0,0,0}	{0,0,-6,28,116,-10,0,0}	{0,0,4,40,62,22,0,0}
14/16	{0,0,-4,18,122,-10,2,0}	{0,0,4,36,62,26,0,0}	{-2,4,-6,16,124,-12,6,-2}	{0,0,0,16,112,0,0,0}	{0,0,-4,18,122,-8,0,0}	{0,0,4,36,62,26,0,0}
15/16	{0,0,-2,8,126,-6,2,0}	{0,0,2,34,62,28,2,0}	{0,2,-2,8,126,-6,2,-2}	{0,0,0,8,120,0,0,0}	{0,0,-2,8,126,-4,0,0}	{0,0,2,34,62,30,0,0}



DAV1D: сокращение набора коэффициентов интерполяции



DAV1D: 4-tap интерполяция

Идея: сократить количество фильтров в наборе до 4.

Для этого необходимо сохранить баланс, пример:

- 1. { -2, 6, -12, 40, 40, -12, 6, -2 }; sum = 64
- 2. { -12, 40, 40, -12 }; sum = 56
- 3. { -12, 44, 44, -12 }; sum = 64

Результат оптимизации: -33.1% времени декодирования

DAV1D: 4-таp интерполяция



DAV1D: интерполяция без дробного остатка

Интерполированный пиксель в одной плоскости =

$$(A * X [0] + B * X [1] \dots + (1 \ll 7) \gg 1) \gg 7$$

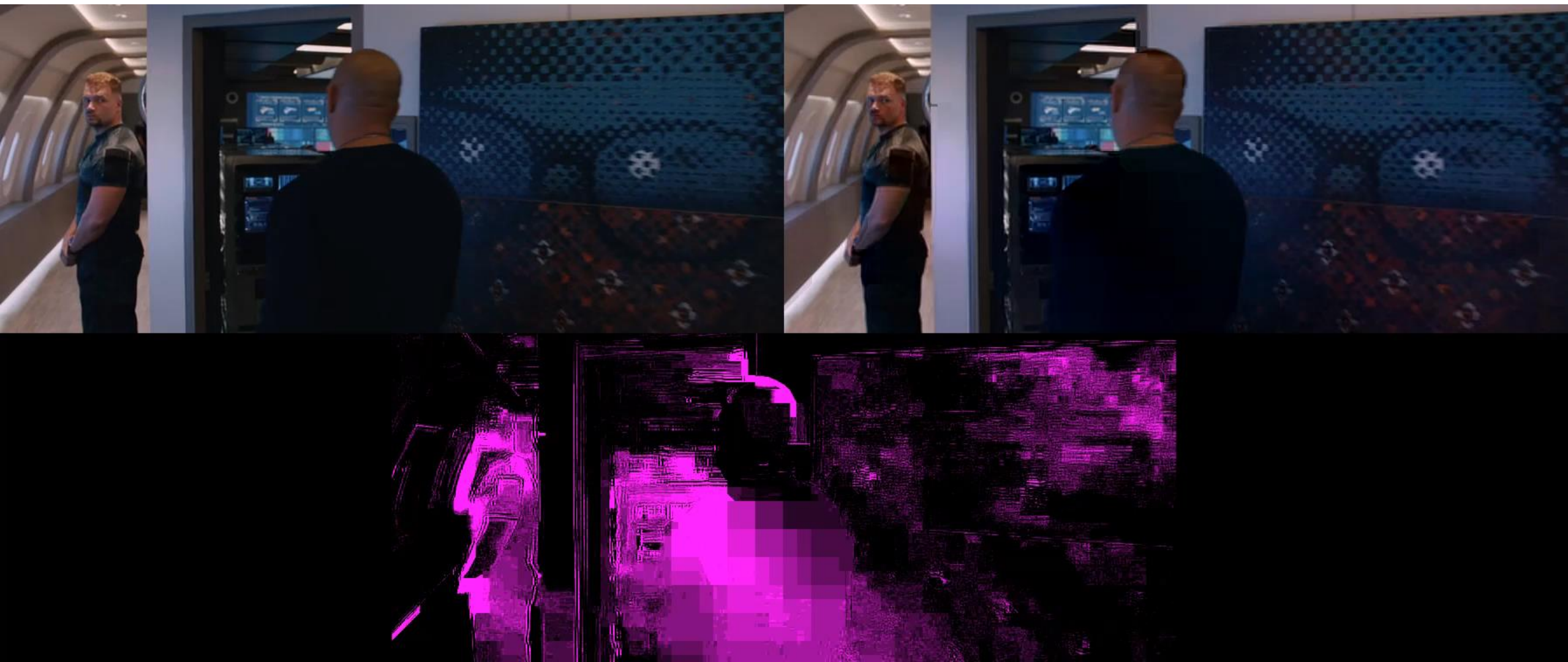
Идея: упростить вычисление интерполяции, вырезав дробный остаток $(1 \ll 7) \gg 1$.

Результат: -36% времени декодирования

DAV1D: интерполяция без дробного остатка

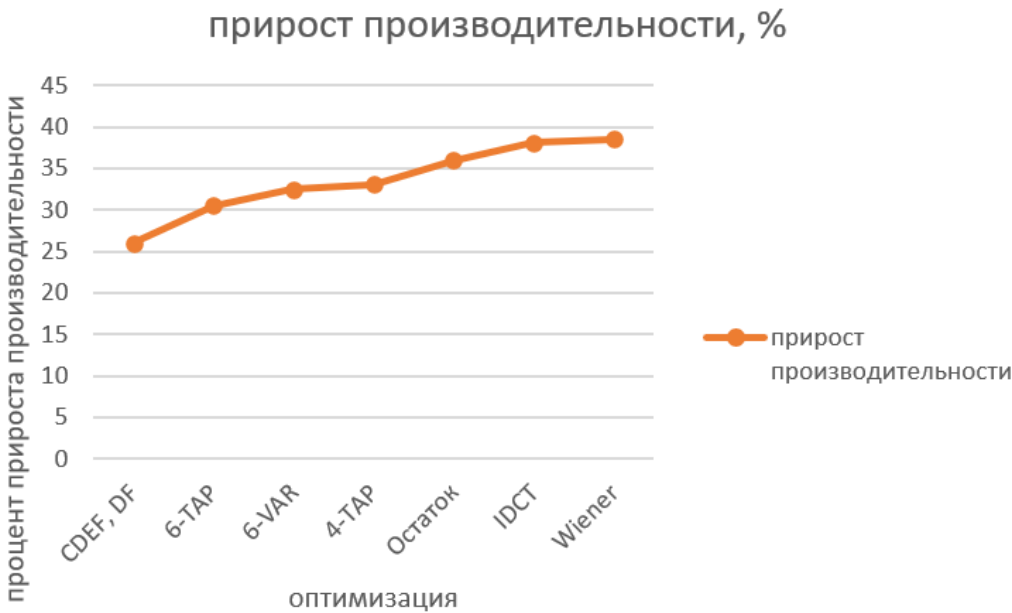


DAV1D: реконструкция без фильтра винера



Результаты оптимизации DAV1D

В результате вышеприведённых оптимизаций время декодирования составило 61.5% от оригинальной версии, это означает что скорость декодирования уменьшилась на 38.5%, что удовлетворяет поставленной цели.



Реализация: оптимизированная
система

Decoder api

Основная функция: Upload

Использует оптимизированный DAV1D

```

25 async def _process_upload(
26     file: UploadFile,
27     token: str,
28     otp_repo: OtpRepository,
29     video_service: VideoService,
30     status_repo: StatusRepository,
31     in_memory: bool = True,
32 ) -> Response:
33     payload = await otp_repo.consume_upload_token(token)
34     await status_repo.create_video('unknown', payload['archive_id'])
35     file_bytes = await file.read()
36     frame_bytes = await video_service.decode_first_frame(file_bytes, in_memory=in_memory)
37     js, _ = await js_connect()
38     try:
39         await js.add_stream(name="NOTIFS", subjects=["notifications"])
40     except Exception:
41         pass
42     await js.publish(
43         "notifications",
44         payload=json.dumps({"user_id": payload["user_id"], "payload": "123 test"}).encode(),
45     )
46
47     return Response(content=frame_bytes, media_type="image/jpeg")
48
49
50 @router.post("/upload")
51 async def upload_video(
52     file: UploadFile = File(...),
53     token: str = Query(...),
54     otp_repo: OtpRepository = Depends(get_otp_repo),
55     video_service: VideoService = Depends(get_video_service),
56     status_repo: StatusRepository = Depends(get_status_repo)
57 ):
58     return await _process_upload(file, token, otp_repo, video_service, status_repo, in_memory=True)
59
60
61 @router.post("/upload_disk")
62 async def upload_video_disk(
63     file: UploadFile = File(...),
64     token: str = Query(...),
65     otp_repo: OtpRepository = Depends(get_otp_repo),
66     video_service: VideoService = Depends(get_video_service),
67     status_repo: StatusRepository = Depends(get_status_repo)
68 ):
69     return await _process_upload(file, token, otp_repo, video_service, status_repo, in_memory=False)

```

decoder-api

core

config.py

security.py

db

dependencies.py

reporter

__init__.py

reporter.py

repositories

otp_repository.py

status_repository.py

user_repository.py

routers

auth.py

user.py

video.py

services

video_service.py

utils

Dockerfile

__init__.py

__main__.py

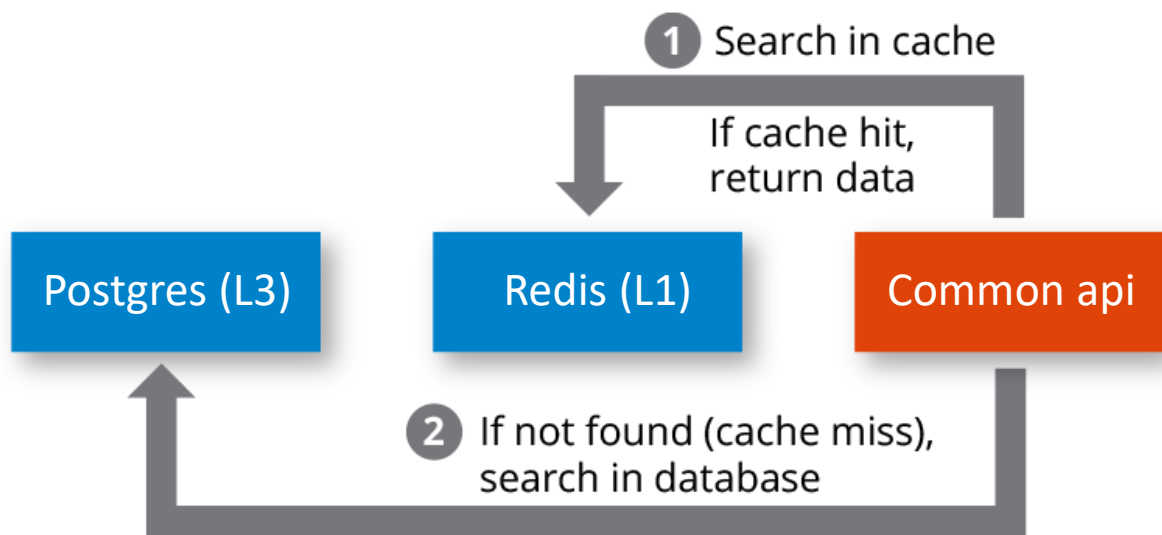
main.py

requirements.txt

Common api

Апи для авторизации, бизнес функций и прочего.

Оптимизация: cache-miss-like авторизация (redis, postgresql)



```
81 async def get_current_user(  
82     credentials: HTTPAuthorizationCredentials = Depends(security),  
83     db: AsyncSession = Depends(get_db),  
84 ):  
85     token = credentials.credentials  
86  
87     # 1. Сначала смотрим в Redis (быстрый путь)  
88     cached_user_id = await redis_client.get(f"access:{token}")  
89     if cached_user_id:  
90         # Достаём username из второго ключа  
91         username = await redis_client.get(f"user_id:{cached_user_id}:username")  
92         if username:  
93             return {"id": int(cached_user_id), "username": username}  
94  
95     # 2. Пропав кэша — идём в БД  
96     result = await db.execute(  
97         select(User).where(User.access_token == token)  
98     )  
99     user = result.scalar_one_or_none()  
100     if not user:  
101         raise HTTPException(status_code=401, detail="Invalid or expired token")  
102  
103     # Прогреваем кэш для следующих запросов  
104     await redis_client.setex(f"access:{token}", ACCESS_TOKEN_TTL, user.id)  
105     await redis_client.setex(f"user_id:{user.id}:username", ACCESS_TOKEN_TTL, user.name)  
106  
107     return {"id": user.id, "username": user.name}  
108
```

common-api
Dockerfile
__init__.py
main.py
requirements.txt

Gateway

- Отказ от прямого проксирования Upload
 - Вместо этого: GET чтобы получить прямую ссылку с одноразовым токеном
- Добавление пороговых значений для загрузки CPU, RAM, DISK MEM: 90%
- Получение информации о загрузке через /ws/replica
- Уведомление клиентов о конце операции через пул вебсокетов на /ws/notify

Frontend

Представляет из себя 2 модуля: FFMPEG WASM и REACT

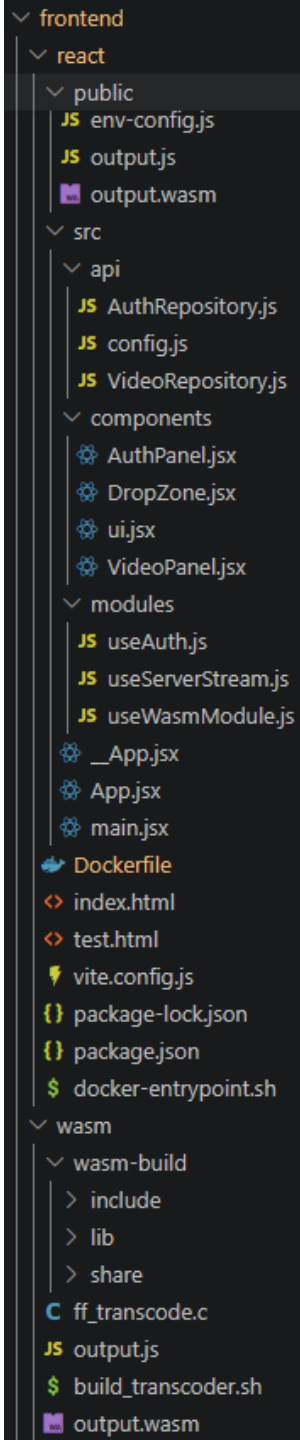
Для компиляции в WASM через Emscripten:

- Были вручную вырезаны функции для обратных трансформаций
- FFMPEG был скомпилирован с флагом `-Oz` для максимальной оптимизации размера
- Было отключено всё кроме муксера и демуксера AV1, а также необходимых протоколов

Результат оптимизации: суммарный размер WASM FFMPEG упал с 20мб до 2мб.

Итоговая конфигурация:

```
emconfigure ./configure --extra-cflags="-Oz -ffunction-sections -fdata-sections -fPIC" --extra-ldflags="-Wl,--gc-sections -Wl,--strip-all, --strip-debug --closure 1" --prefix=$(pwd)/wasm-build --disable-programs
--disable-shared --disable-doc --enable-cross-compile --disable-sdl2 --disable-iconv --
disable-everything --enable-protocols --enable-demuxers --enable-muxer=obu --enable-parser=av1
--enable-static --enable-hardcoded-tables --enable-avcodec --enable-avutil --enable-avformat --
cc=emcc --target-os=none --disable-x86asm --disable-inline-asm --arch=x86_64 --disable-d3d11va
--disable-d3d12va --disable-dxva2 --disable-cuda-llvm --disable-audiotoolbox --disable-amf --
disable-nvdec --disable-nvenc --disable-large-tests --disable-asm --disable-xlib --enable-small
--disable-debug --disable-dwt --disable-hwaccels --disable-txtpages --disable-network --disable-
pthreads --disable-os2threads --disable-w32threads --disable-optimizations --enable-pic
```



Развертывание

Весь проект имеет 2 варианта развертывания: через Docker swarm и через Docker compose. При первом варианте реплики декодера самостоятельно устанавливаются на сервера, по одной на сервер включая узел-менеджер.

На данный момент проект развернут с помощью Docker swarm на двух серверах.

Результаты проекта

Результат оптимизации:

Оптимизированная система работает быстрее наивной на 52.1%

- В качестве метрики использовано среднее время 5 итераций от нажатия кнопки «загрузить» до момента конца декодирования в обеих системах

Код проекта

Оптимизированный декодер

WASM FFMPEG

Снапшот текущего состояния
проекта

Развернутый проект

<https://github.com/ressiwage/DAV2D>

<https://github.com/ressiwage/FFMPEG>

<https://github.com/ressiwage/UFAV1VDSS7>

<http://194.87.131.81:4173/>

